República Bolivariana de Venezuela

Ministerio del Poder Popular para la Educación

Universidad Católica Andrés Bello

Computación en la Nube

Profesor: Jorge Rojas

Arquitectura del Software

Sistema de Citas a Ciegas en Línea *"Esencia"*

INTEGRANTES

Matias Silveira - C.I. 31.020.586
Andrea Mendoza - C.I. 30.032.234
Daniela Silva - C.I. 30.225.188
Rosxana Medero - C.I. 29.969.634
Samuel Velásquez - C.I. 29.966.921

09 de octubre de 2026

Índice

1. Resumen de decisiones arquitectónicas..... 3
2. La arquitectura..... 5
3. Diagrama de arquitectura inicial..... 6
4. Explicación de cada componente..... 7
4.1 Frontend — SPA..... 7
4.2 Backend / API — contenedor Docker en Render..... 7
4.3 Persistencia — PostgreSQL en Supabase..... 7
4.4 Caché — Redis en Upstash..... 7
4.5 Servicios externos — Sentry, monitor de uptime, GHCR..... 7
5. Cómo se comunican las capas..... 8
6. Modelos de servicio cloud..... 9

1. Resumen de Decisiones Arquitectónicas

A continuación, se contempla el resumen ejecutivo de las decisiones arquitectónicas del sistema *Esencia* en la tabla anexa. Cada decisión se identifica como ADR (Architecture Decision Record) y su número correspondiente.

#	Decisión	Justificación	Alternativa principal descartada
ADR-0002	Monolito modular	Un solo backend Express organizado en módulos de dominio.	Microservicios: costo de orquestación injustificado para el volumen y el tamaño del equipo
ADR-0003	React + Vite + TypeScript (frontend) y Node + Express + TypeScript (backend)	Un solo lenguaje (TypeScript) en todo el stack	NestJS: capa de abstracción adicional que no aporta al tamaño de este proyecto
ADR-0004	PostgreSQL en Supabase como base de datos	Relacional porque hay restricciones de integridad reales (likes/matches únicos)	MongoDB: reimplementaría esas restricciones a nivel de aplicación
ADR-0005	Socket.IO para el chat en tiempo real	WebSocket con reconexión automática y salas por match	Polling HTTP: latencia alta y carga innecesaria
ADR-0006	Redis en Upstash como caché	Cache-aside del feed, rate limiting distribuido, adaptador de Socket.IO	No usar caché: se renuncia a una mejora real ya justificada
ADR-0007	Vercel + Render (Docker)	Nube pública multiproveedor, cada capa gratuita en el proveedor más simple para esa función	AWS/Azure/GCP completos: carga de configuración (VPC, IAM) sin beneficio proporcional
ADR-0008	JWT propio (access + refresh)	Autenticación implementada por el equipo, no delegada	Supabase Auth / Auth0: dificulta explicar el mecanismo en la defensa individual
ADR-0009	Sin almacenamiento de imágenes	Decisión de producto con impacto arquitectónico: elimina una capa entera (object storage/CDN de medios)	Foto de perfil opcional: contradice la propuesta de valor

#	Decisión	Justificación	Alternativa principal descartada
ADR-0010	CI/CD con GitHub Actions + GHCR	Un solo proveedor (GitHub) para código, CI, CD y registro de imágenes	Herramientas externas de CI: integración cruzada innecesaria
ADR-0011	pino + Sentry + health checks + monitor de uptime	Cuatro mecanismos complementarios, todos en capa gratuita	Stack propio (Prometheus/Grafana): infraestructura adicional injustificada
ADR-0012	Monorepo sin workspaces + GitHub Flow	apps/api y apps/web con build independiente; main siempre desplegable	GitFlow completo: ceremonia innecesaria para 5 personas y sprints de 2 semanas

Fuente: elaboración propia.

2. La Arquitectura

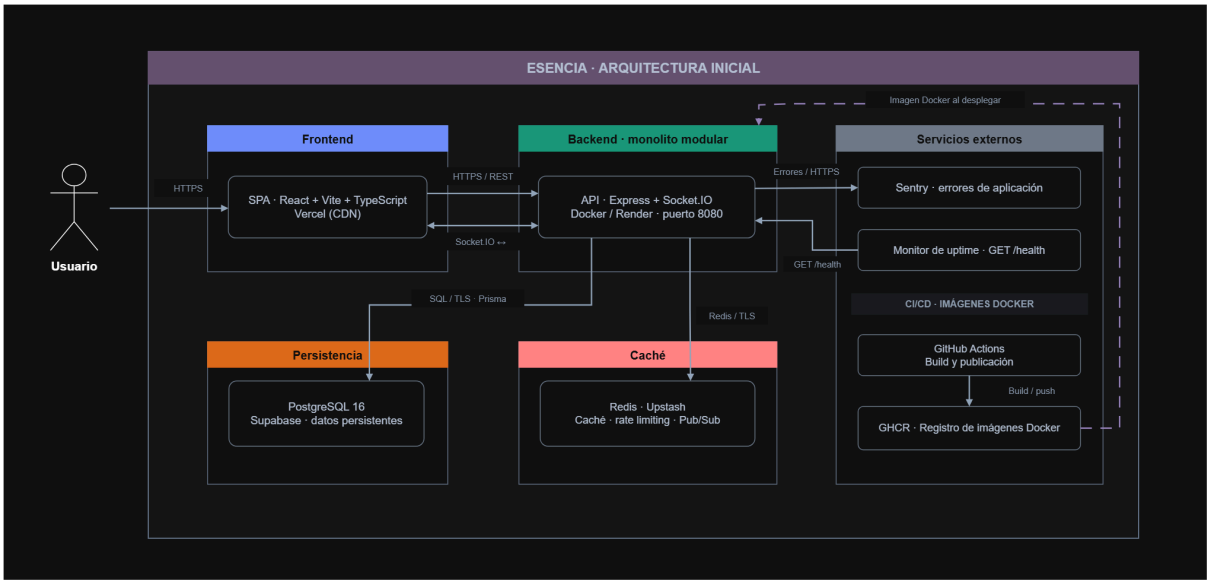
A continuación se define en forma de tabla la arquitectura definida para el proyecto, identificado por capas:

Capa	Componente	Tecnología	Proveedor / modelo cloud
Frontend	SPA	React + Vite + TypeScript + Tailwind	Vercel — PaaS (hosting estático + CDN)
Backend / API	Servicio Express	Node.js 20 + Express + TypeScript + Socket.IO, Prisma ORM, Zod	Render — PaaS con contenedores en Docker
Persistencia	Base de datos relacional	PostgreSQL 16	Supabase — DBaaS
Caché	Almacén en memoria	Redis	Upstash — DBaaS/SaaS serverless
Servicios externos	Observabilidad y CI/CD	Sentry, monitor de uptime, GitHub Actions + GHCR	SaaS

Fuente: elaboración propia

No existe una capa de almacenamiento de archivos/imágenes, acatando a las decisiones de arquitectura y a la naturaleza del sistema planteado.

3. Diagrama de Arquitectura Inicial



Fuente: elaboración propia

4. Explicación de Cada Componente

4.1 Frontend — SPA

React + Vite + TypeScript, servida como archivos estáticos desde el CDN global de Vercel. No ejecuta lógica de negocio, solo renderiza la interfaz, gestiona el estado de sesión (el *access token* vive en memoria del navegador) y consume la API. Es completamente stateless del lado del servidor, cualquier instancia del CDN puede servir cualquier solicitud.

4.2 Backend / API — contenedor Docker en Render

El único componente con estado de proceso. Mantiene las conexiones WebSocket abiertas para el chat. Expone dos superficies sobre el mismo puerto (8080): REST (/api/v1/*) y WebSocket (Socket.IO). Internamente se organiza en módulos de dominio (auth, users, prompts, discovery, likes-matches, chat, safety, health), cada uno con sus rutas, su lógica de negocio y su acceso a datos vía Prisma. Para HTTP es stateless; para WebSocket usa el adaptador de Redis para poder escalar a varias réplicas sin perder mensajes.

4.3 Persistencia — PostgreSQL en Supabase

Fuente de persistencia del sistema. Se accede siempre a través del connection pooler de Supabase (pgbouncer, puerto 6543), nunca por conexión directa, para no agotar el límite de conexiones concurrentes del plan gratuito. Es stateful por definición es la única capa donde un fallo no tiene fallback silencioso.

4.4 Caché — Redis en Upstash

Esta capa cuenta con tres usos concretos:

1. Cache-aside del feed de descubrimiento (TTL 5 min, invalidado al dar like/pass)
2. Contador de *rate limiting* distribuido
3. Adaptador pub/sub de Socket.IO para sincronizar mensajes entre réplicas del backend. Es stateful pero desechable: si se vacía, el sistema sigue funcionando porque todo se puede recalcular desde Postgres.

4.5 Servicios externos — Sentry, monitor de uptime, GHCR

Fuera del camino crítico de cualquier solicitud de usuario: si cualquiera de los tres falla, el sistema sigue funcionando, aunque se pierda observabilidad o capacidad de desplegar. Sentry captura errores no controlados en tiempo de

ejecución. El monitor de uptime hace ping periódico a /health, lo que además mitiga el arranque en frío de Render. GHCR almacena las imágenes Docker que construye GitHub Actions y que Render despliega.

5. Cómo se Comunican las Capas

Origen → Destino	Protocolo	Detalle
Navegador → Frontend	HTTPS	Sirve la SPA compilada desde el CDN
Frontend → Backend (REST)	HTTPS/JSON	Authorization: Bearer <accessToken>
Frontend → Backend (tiempo real)	WSS (Socket.IO)	JWT verificado en el *handshake* de conexión
Backend → Persistencia	TLS · SQL vía Prisma	A través del *pooler* de Supabase, puerto 6543
Backend → Caché	TLS · comandos Redis	Cliente ioredis
Backend → Sentry	HTTPS	Solo en caso de error no controlado

Fuente: elaboración propia

6. Modelos de Servicio Cloud

Componente	Proveedor	Modelo
Frontend	Vercel	PaaS
Backend/API	Render	PaaS con contenedores
Base de datos	Supabase	DBaaS
Caché	Upstash	DBaaS/SaaS serverless
CI/CD	GitHub Actions	SaaS
Registro de imágenes	GHCR	SaaS
Observabilidad	Sentry + uptime monitor	SaaS

Fuente: elaboración propia

No se usa IaaS (no hay motivo para administrar un SO) ni FaaS (el backend mantiene conexiones WebSocket persistentes, incompatible con funciones de invocación corta). El modelo de despliegue se basa en una nube pública multiproveedor.